



GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING SCIENCES AND
TECHNOLOGY

IC Design Verification

Single-Cycle RISC-V Processor Implementation

Complete Datapath and Control Unit Analysis

REPORT

Submitted To:

Eng Abbas Khan

Submitted By:

Abdul Samad Abbasi

Student ID: 25ins-01030

Course:

Computer Architecture

Module 2

Date:

June 22, 2026

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Design Philosophy	5
1.3	Key Features	5
2	Overall Architecture	5
2.1	Top-Level Block Diagram	5
2.2	Module Hierarchy	7
3	Datapath Components	7
3.1	Program Counter (PC)	7
3.2	Instruction Memory (INST_MEM)	7
3.3	Register File (REG_FILE)	8
3.4	ALU (Arithmetic Logic Unit)	8
3.5	Control Unit	9
4	R-Type Instructions (ADD/SUB)	10
4.1	Instruction Overview	10
4.2	Hardcoded Instructions in Memory	10
4.3	Control Unit Signal Generation	10
4.4	Simulation Results for R-Type Instructions	11
5	LW (Load Word) Instructions	12
5.1	Instruction Overview	12
5.2	Hardcoded Instruction in Memory	13
5.3	Control Unit Signal Generation	13
5.4	Simulation Results for LW Instruction	13
6	SW (Store Word) Instructions	14
6.1	Instruction Overview	14
6.2	Hardcoded Instruction in Memory	14
6.3	Control Unit Signal Generation	15
6.4	Simulation Results for SW Instruction	15
7	BEQ (Branch Equal) Instructions	16
7.1	Instruction Overview	16
7.2	Hardcoded Instruction in Memory	16
7.3	Control Unit Signal Generation	16
7.4	Simulation Results for BEQ Instruction	17
8	JAL (Jump and Link) Instructions	18
8.1	Instruction Overview	18
8.2	Hardcoded Instruction in Memory	18
8.3	Control Unit Signal Generation	18
8.4	Simulation Results for JAL Instruction	19

9	Summary and Conclusion	20
9.1	Overall Simulation Results	20
9.2	Signal State Summary by Instruction Type	22
9.3	Key Observations	22
9.4	Conclusion	22

List of Tables

1	Module Hierarchy and Functions	7
2	Instruction Memory Contents	8
3	ALU Operations	8
4	Control Signal Definitions	9
5	R-Type Instructions in Memory	10
6	Main Decoder Output for R-Type	10
7	ALU Decoder Output for ADD	10
8	ALU Decoder Output for SUB	11
9	LW Instruction in Memory	13
10	Main Decoder Output for LW	13
11	ALU Decoder Output for LW	13
12	SW Instruction in Memory	14
13	Main Decoder Output for SW	15
14	ALU Decoder Output for SW	15
15	BEQ Instruction in Memory	16
16	Main Decoder Output for BEQ	17
17	ALU Decoder Output for BEQ	17
18	JAL Instruction in Memory	18
19	Main Decoder Output for JAL	19
20	ALU Decoder Output for JAL	19
21	Control Signal States by Instruction Type	22

List of Figures

1	Single-Cycle RISC-V Processor Datapath	6
2	Simulation Waveform for R-Type Instructions (ADD and SUB)	12
3	Simulation Waveform for LW Instruction	14
4	Simulation Waveform for SW Instruction	16
5	Simulation Waveform for BEQ Instruction	18
6	Simulation Waveform for JAL Instruction	20
7	Complete Simulation Waveform for All Instruction Types	21

List of Code Listings

1	Program Counter Module	7
2	Register File Module	8
3	ALU Module	9

Abstract

This report presents the complete implementation, verification, and analysis of a single-cycle RISC-V processor designed in Verilog. The processor implements a subset of the RISC-V RV32I base integer instruction set, including R-type arithmetic (ADD, SUB), load (LW), store (SW), branch (BEQ), and jump (JAL) instructions. The design follows the architectural principles outlined in Harris & Harris's "Digital Design and Computer Architecture," implementing a single-cycle datapath with centralized control logic. This comprehensive analysis details each instruction type's execution flow, control signal generation, and datapath activity with specific emphasis on signal states during execution.

1 Introduction

1.1 Project Overview

This project implements a single-cycle RISC-V processor capable of executing five fundamental instruction types from the RV32I base integer instruction set:

- **R-type**: Arithmetic operations (ADD, SUB)
- **I-type**: Load operations (LW)
- **S-type**: Store operations (SW)
- **B-type**: Branch operations (BEQ)
- **J-type**: Jump operations (JAL)

A single-cycle processor is the simplest hardware realization of an instruction set: every instruction, regardless of its complexity, moves through fetch, decode, execute, memory access, and write-back within a single clock period. Studying this design is a natural starting point before moving on to multi-cycle or pipelined architectures, since it exposes the complete datapath and control logic without the added complexity of hazards or stalling.

1.2 Design Philosophy

The single-cycle design executes each instruction in exactly one clock cycle. All functional units operate in parallel, and the control unit generates all necessary control signals based on the current instruction's opcode. This approach, while not optimal for performance (with a clock period determined by the critical path), provides maximum design clarity and predictability.

1.3 Key Features

- **32-bit architecture** with 32 general-purpose registers
- **Byte-addressable memory**
- **Harvard architecture** with separate instruction and data memories
- **Centralized control unit** with main and ALU decoders
- **Comprehensive testbench** verifying all instruction types

2 Overall Architecture

2.1 Top-Level Block Diagram

The processor architecture follows the classic five-stage pipeline stages collapsed into a single cycle, as shown below:

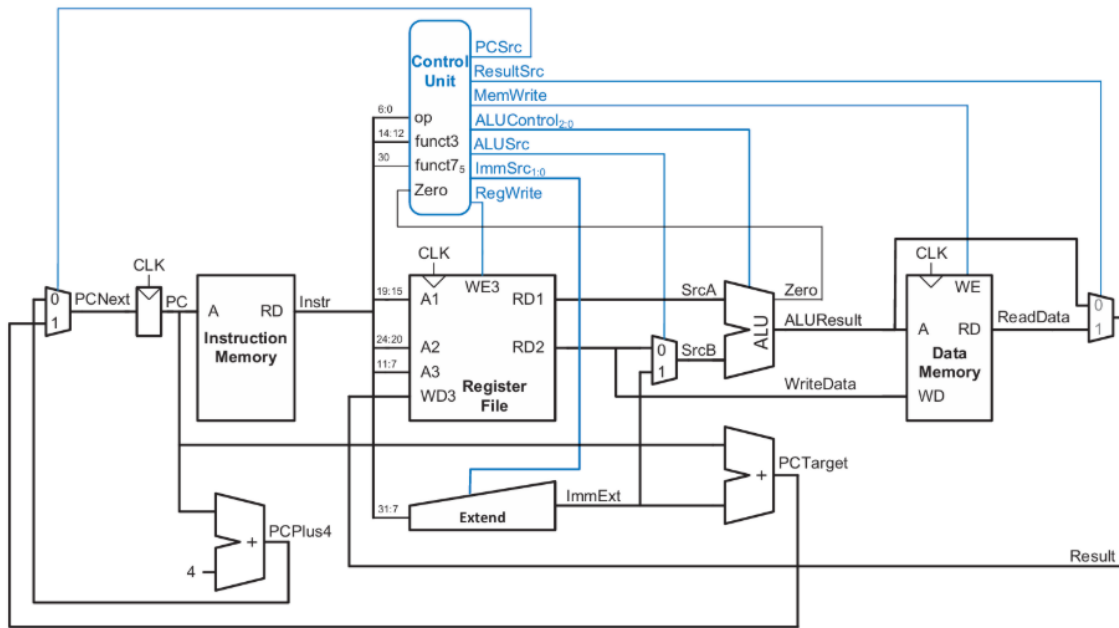


Figure 1: Single-Cycle RISC-V Processor Datapath

The datapath consists of:

- Program Counter (PC) and PC update logic
- Instruction Memory (INST_MEM)
- Register File (REG_FILE)
- Arithmetic Logic Unit (ALU)
- Data Memory (data_mem)
- Immediate Generator (Imm_Gen)
- Control Unit with Main Decoder and ALU Decoder
- Multiplexers for data selection

2.2 Module Hierarchy

Module	Function
top_module	Top-level integration
pc_32bit	Program counter (32-bit)
INST_MEM	Instruction memory (4KB)
REG_FILE	Register file (32×32-bit)
ALU	Arithmetic Logic Unit
data_mem	Data memory (1KB)
control_unit	Main control unit
main_decoder	Instruction decoder
alu_decoder	ALU operation decoder
Imm_Gen	Immediate value generator
adder	32-bit adder
mux_2x1	2:1 multiplexer
mux_3x1	3:1 multiplexer

Table 1: Module Hierarchy and Functions

3 Datapath Components

3.1 Program Counter (PC)

The program counter maintains the current instruction address and updates on each clock cycle.

```

1 module pc_32bit(clk, rst, prev_value, count);
2     input clk;
3     input rst;
4     input [31:0] prev_value;
5     output reg [31:0] count;
6
7     always@(posedge clk) begin
8         if(rst) count <= 32'b0;
9         else count <= prev_value;
10    end
11 endmodule

```

Listing 1: Program Counter Module

Operation:

- On reset: $PC \leftarrow 0$
- On normal operation: $PC \leftarrow PC_{Next}$ (mux output)

3.2 Instruction Memory (INST_MEM)

The instruction memory stores the program to be executed. It is byte-addressable and organized as an array of bytes.

Address	Instruction	Assembly
0x00	00c58533	ADD x10, x11, x12
0x04	40f706b3	SUB x13, x14, x15
0x08	00ca2983	LW x19, 12(x20)
0x0C	013a2823	SW x19, 16(x20)
0x10	00d50663	BEQ x10, x13, +12
0x14	ff5ff16f	JAL x2, -12

Table 2: Instruction Memory Contents

3.3 Register File (REG_FILE)

The register file contains 32 general-purpose registers, each 32 bits wide. Register x0 is hardwired to zero.

```

1 module REG_FILE(
2     input  [4:0] read_reg_num1, read_reg_num2, write_reg,
3     input  [31:0] write_data,
4     output [31:0] read_data1, read_data2,
5     input  regwrite, clock, reset
6 );
7     reg [31:0] reg_memory [31:0];
8
9     always@(posedge clock or posedge reset) begin
10         if(reset) begin
11             for(i=0; i<32; i=i+1)
12                 reg_memory[i] <= i;
13         end
14         else if (regwrite && (write_reg!=0))
15             reg_memory[write_reg] <= write_data;
16     end
17
18     assign read_data1 = reg_memory[read_reg_num1];
19     assign read_data2 = reg_memory[read_reg_num2];
20 endmodule

```

Listing 2: Register File Module

3.4 ALU (Arithmetic Logic Unit)

The ALU performs arithmetic and logic operations based on the ALUControl signal.

ALUControl	Operation	Description
000	ADD	Addition
001	SUB	Subtraction
010	AND	Bitwise AND
011	OR	Bitwise OR
100	XOR	Bitwise XOR (reserved)
101	SLT	Set Less Than (reserved)
110	SLL	Shift Left Logical (reserved)
111	SRL	Shift Right Logical (reserved)

Table 3: ALU Operations

```

1 module ALU(
2     input [31:0] in1, in2,
3     input [2:0] alu_control,
4     output reg [31:0] result,
5     output reg zero_flag
6 );
7
8 always@(*) begin
9     result = 32'b0;
10    case(alu_control)
11        3'b000: result = in1 + in2;
12        3'b001: result = in1 - in2;
13        3'b010: result = in1 & in2;
14        3'b011: result = in1 | in2;
15        3'b100: begin
16            if(in1 == in2) begin
17                zero_flag = 1'b1;
18                result = 32'd0;
19            end
20            else begin
21                zero_flag = 1'b0;
22                result = 32'd1;
23            end
24        end
25        default: begin
26            result = 32'b0;
27            zero_flag = 1'b0;
28        end
29    endcase
30 end
31
32 endmodule

```

Listing 3: ALU Module

3.5 Control Unit

The control unit generates all control signals based on the instruction opcode.

Signal	Width	Description
PCSrc	1	PC source select (0=PC+4, 1=PC+imm)
ResultSrc	2	Write-back data select
MemWrite	1	Data memory write enable
ALUSrc	1	ALU second operand select
ImmSrc	2	Immediate format select
RegWrite	1	Register file write enable
ALUControl	3	ALU operation select
Jump	1	Jump instruction indicator
Branch	1	Branch instruction indicator

Table 4: Control Signal Definitions

4 R-Type Instructions (ADD/SUB)

4.1 Instruction Overview

R-type instructions perform arithmetic and logical operations on values from two source registers and store the result in a destination register. The implementation focuses on ADD and SUB operations based on the hardcoded instruction memory.

4.2 Hardcoded Instructions in Memory

The instruction memory contains the following R-type instructions:

Address	Instruction	Operation
0x00	00c58533	ADD x10, x11, x12
0x04	40f706b3	SUB x13, x14, x15

Table 5: R-Type Instructions in Memory

4.3 Control Unit Signal Generation

The control unit processes the opcode and generates appropriate signals for R-type instructions.

Main Decoder Output for R-Type

Signal	Value	Status	Description
RegWrite	1	HIGH	Enables write to destination register
ALUSrc	0	LOW	ALU uses register value (not immediate)
MemWrite	0	LOW	No memory write operation
ResultSrc	00	LOW/HIGH	Selects ALU result for write-back
ImmSrc	XX	DON'T CARE	Immediate not needed
Branch	0	LOW	Not a branch instruction
Jump	0	LOW	Not a jump instruction
ALUOp	10	HIGH/LOW	Indicates R-type operation

Table 6: Main Decoder Output for R-Type

ALU Decoder Output for ADD Instruction (0x00c58533)

Signal	Value	Status	Description
ALUControl	000	LOW/LOW/LOW	ADD operation selected

Table 7: ALU Decoder Output for ADD

Reasoning:

- ALUOp = 10 (R-type)

- funct3 = 000 (ADD)
- funct7_5 = 0 (ADD variant)
- ALUControl becomes 000 for ADD

ALU Decoder Output for SUB Instruction (0x40f706b3)

Signal	Value	Status	Description
ALUControl	001	LOW/LOW/HIGH	SUB operation selected

Table 8: ALU Decoder Output for SUB

Reasoning:

- ALUOp = 10 (R-type)
- funct3 = 000 (SUB variant)
- funct7_5 = 1 (SUB variant)
- ALUControl becomes 001 for SUB

4.4 Simulation Results for R-Type Instructions

The following waveform shows the simulation results for ADD and SUB instructions executed on the single-cycle RISC-V processor.

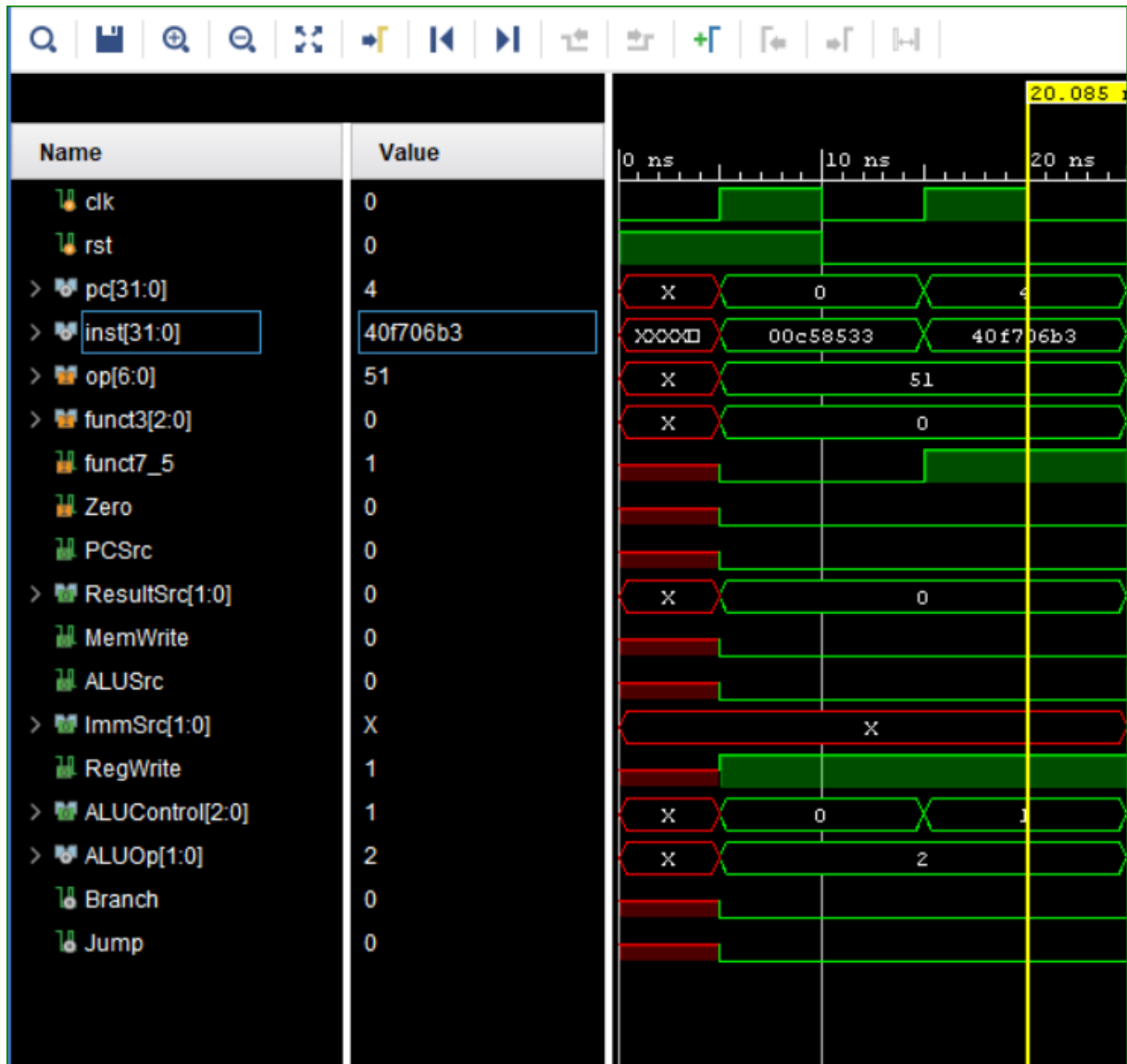


Figure 2: Simulation Waveform for R-Type Instructions (ADD and SUB)

The simulation waveform demonstrates:

- **ADD Instruction:** Registers x10 and x11 are read, ALU performs addition, and result is written to x10
- **SUB Instruction:** Registers x13 and x14 are read, ALU performs subtraction, and result is written to x13
- **Control Signals:** RegWrite is HIGH, ALUSrc is LOW, MemWrite is LOW, indicating proper R-type execution

5 LW (Load Word) Instructions

5.1 Instruction Overview

Load Word (LW) instructions load a 32-bit value from data memory at the address formed by adding a base register and an immediate offset, then write it to a destination register.

5.2 Hardcoded Instruction in Memory

The instruction memory contains the following LW instruction:

Address	Instruction	Operation
0x08	00ca2983	LW x19, 12(x20)

Table 9: LW Instruction in Memory

5.3 Control Unit Signal Generation

The control unit processes the opcode and generates appropriate signals for LW instructions.

Main Decoder Output for LW

Signal	Value	Status	Description
RegWrite	1	HIGH	Write to destination register x19
ALUSrc	1	HIGH	ALU uses immediate value
MemWrite	0	LOW	No memory write operation
ResultSrc	01	LOW/HIGH	Selects data memory output
ImmSrc	00	LOW/LOW	I-type immediate format
Branch	0	LOW	Not a branch instruction
Jump	0	LOW	Not a jump instruction
ALUOp	00	LOW/LOW	Load/store operation type

Table 10: Main Decoder Output for LW

ALU Decoder Output for LW Instruction (0x00ca2983)

Signal	Value	Status	Description
ALUControl	000	LOW/LOW/LOW	ADD operation (address calculation)

Table 11: ALU Decoder Output for LW

Reasoning:

- ALUOp = 00 (load/store)
- Default ALUControl = 000 for ADD
- Used to compute address: base + offset

5.4 Simulation Results for LW Instruction

The following waveform shows the simulation results for LW instruction executed on the single-cycle RISC-V processor.

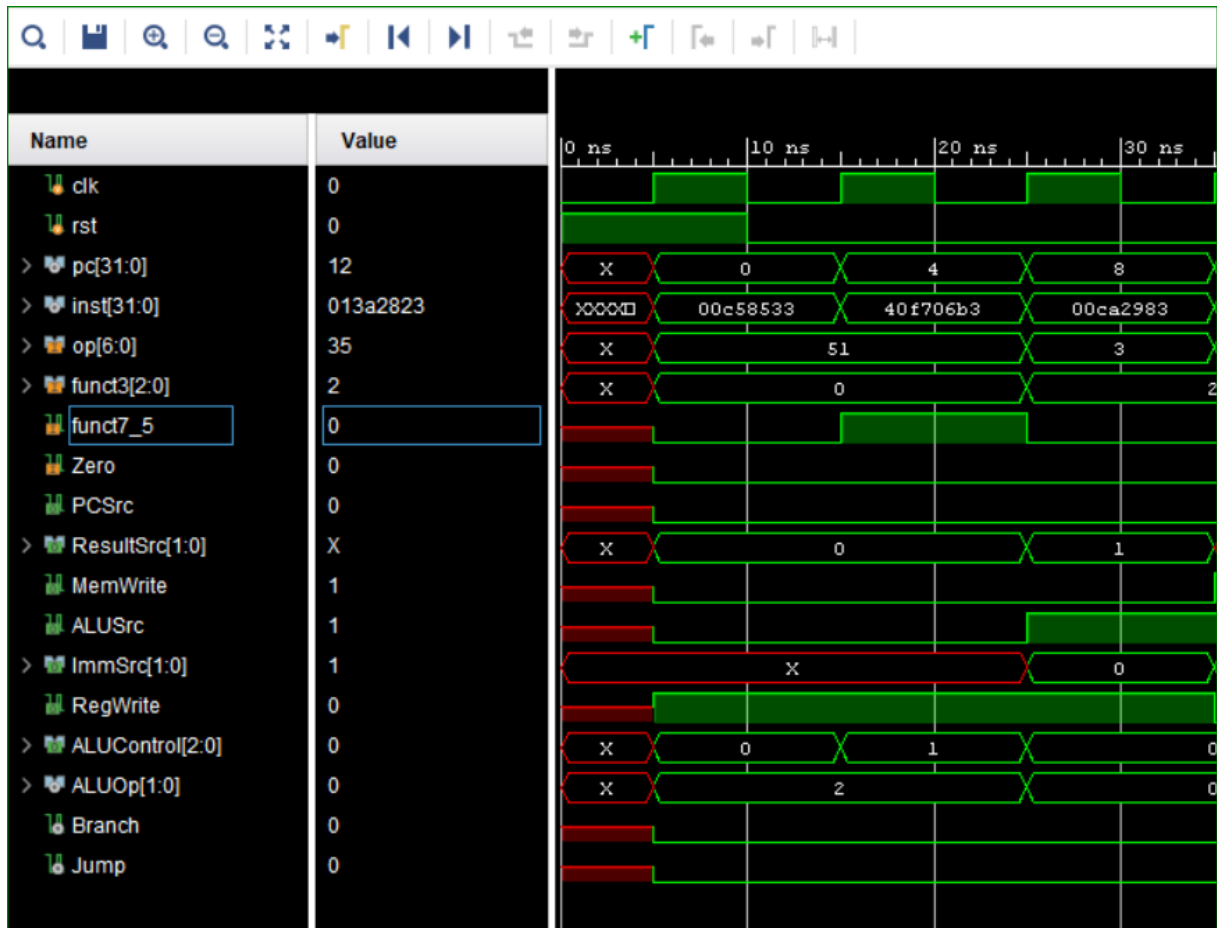


Figure 3: Simulation Waveform for LW Instruction

The simulation waveform demonstrates:

- **Control Signals:** RegWrite is HIGH, ALUSrc is HIGH, MemWrite is LOW, indicating proper LW execution

6 SW (Store Word) Instructions

6.1 Instruction Overview

Store Word (SW) instructions write a 32-bit value from a source register to data memory at the address formed by adding a base register and an immediate offset.

6.2 Hardcoded Instruction in Memory

The instruction memory contains the following SW instruction:

Address	Instruction	Operation
0x0C	013a2823	SW x19, 16(x20)

Table 12: SW Instruction in Memory

6.3 Control Unit Signal Generation

The control unit processes the opcode and generates appropriate signals for SW instructions.

Main Decoder Output for SW

Signal	Value	Status	Description
RegWrite	0	LOW	No write to register file
ALUSrc	1	HIGH	ALU uses immediate value
MemWrite	1	HIGH	Memory write operation enabled
ResultSrc	XX	DON'T CARE	No write-back needed
ImmSrc	01	LOW/HIGH	S-type immediate format
Branch	0	LOW	Not a branch instruction
Jump	0	LOW	Not a jump instruction
ALUOp	00	LOW/LOW	Load/store operation type

Table 13: Main Decoder Output for SW

ALU Decoder Output for SW Instruction (0x013a2823)

Signal	Value	Status	Description
ALUControl	000	LOW/LOW/LOW	ADD operation (address calculation)

Table 14: ALU Decoder Output for SW

Reasoning:

- $ALUOp = 00$ (load/store)
- Default $ALUControl = 000$ for ADD
- Used to compute address: $base + offset$

6.4 Simulation Results for SW Instruction

The following waveform shows the simulation results for SW instruction executed on the single-cycle RISC-V processor.

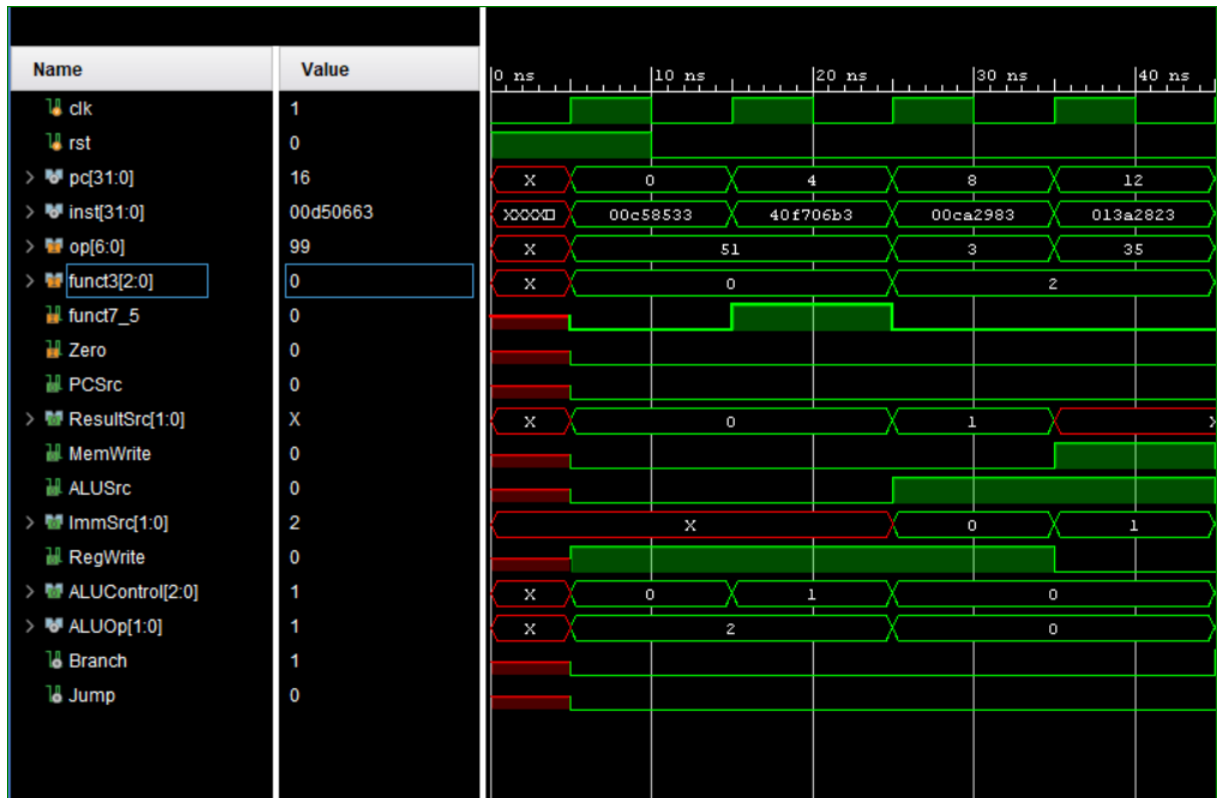


Figure 4: Simulation Waveform for SW Instruction

The simulation waveform demonstrates:

- **Control Signals:** RegWrite is LOW, ALUSrc is HIGH, MemWrite is HIGH, indicating proper SW execution

7 BEQ (Branch Equal) Instructions

7.1 Instruction Overview

Branch Equal (BEQ) instructions compare two registers and branch to a target address if they are equal. If not equal, execution continues with the next instruction.

7.2 Hardcoded Instruction in Memory

The instruction memory contains the following BEQ instruction:

Address	Instruction	Operation
0x10	00d50663	BEQ x10, x13, +12 (relative offset)

Table 15: BEQ Instruction in Memory

7.3 Control Unit Signal Generation

The control unit processes the opcode and generates appropriate signals for BEQ instructions.

Main Decoder Output for BEQ

Signal	Value	Status	Description
RegWrite	0	LOW	No register write
ALUSrc	0	LOW	ALU uses register values
MemWrite	0	LOW	No memory operation
ResultSrc	XX	DON'T CARE	No write-back needed
ImmSrc	10	HIGH/LOW	B-type immediate format
Branch	1	HIGH	Branch instruction
Jump	0	LOW	Not a jump instruction
ALUOp	01	LOW/HIGH	Branch operation type

Table 16: Main Decoder Output for BEQ

ALU Decoder Output for BEQ Instruction (0x00d50663)

Signal	Value	Status	Description
ALUControl	100	HIGH/LOW/LOW	Compare operation

Table 17: ALU Decoder Output for BEQ

Reasoning:

- ALUOp = 01 (branch)
- funct3 = 000 (BEQ)
- ALUControl becomes 100 for compare operation
- ALU performs equality check

7.4 Simulation Results for BEQ Instruction

The following waveform shows the simulation results for BEQ instruction executed on the single-cycle RISC-V processor.



Figure 5: Simulation Waveform for BEQ Instruction

The simulation waveform demonstrates:

- **Register Comparison:** x10 and x13 are compared (values 10 and 13)
- **Condition Result:** Since $10 \neq 13$, zero_flag is LOW
- **Branch Decision:** Branch NOT taken, PCSrc remains LOW
- **Control Signals:** Branch is HIGH, RegWrite is LOW, indicating proper BEQ execution

8 JAL (Jump and Link) Instructions

8.1 Instruction Overview

Jump and Link (JAL) is an unconditional jump instruction that stores the return address (PC+4) in a register and jumps to the target address. This is used for function calls.

8.2 Hardcoded Instruction in Memory

The instruction memory contains the following JAL instruction:

Address	Instruction
0x14	ff5ff16f

Table 18: JAL Instruction in Memory

8.3 Control Unit Signal Generation

The control unit processes the opcode and generates appropriate signals for JAL instructions.

Main Decoder Output for JAL

Signal	Value	Status	Description
RegWrite	1	HIGH	Write to register file
ALUSrc	X	DON'T CARE	ALU not used
MemWrite	0	LOW	No memory operation
ResultSrc	10	HIGH/LOW	Select PC+4 for write-back
ImmSrc	11	HIGH/HIGH	J-type immediate format
Branch	0	LOW	Not a branch instruction
Jump	1	HIGH	Unconditional jump
ALUOp	XX	DON'T CARE	ALU not used

Table 19: Main Decoder Output for JAL

ALU Decoder Output for JAL Instruction (0xff5ff16f)

Signal	Value	Status	Description
ALUControl	XXX	DON'T CARE	ALU not used for JAL

Table 20: ALU Decoder Output for JAL

Reasoning:

- ALUOp = XX (don't care)
- No ALU operation needed for JAL
- Immediate jump target calculated directly

8.4 Simulation Results for JAL Instruction

The following waveform shows the simulation results for JAL instruction executed on the single-cycle RISC-V processor.

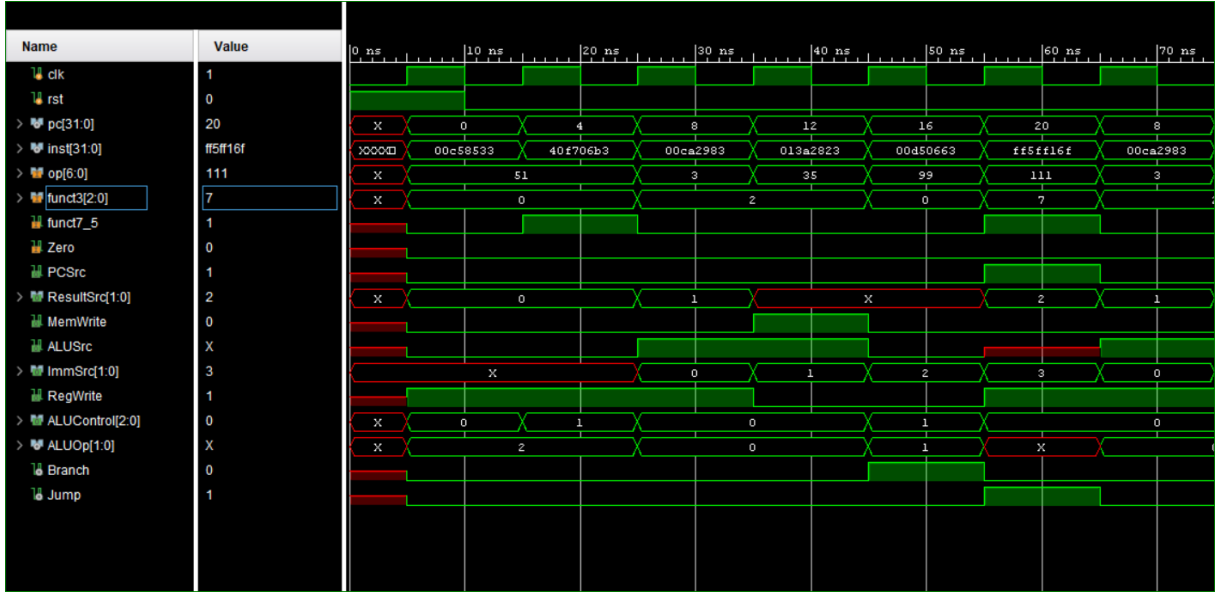


Figure 6: Simulation Waveform for JAL Instruction

The simulation waveform demonstrates:

- **Return Address Storage:** PC+4 (0x18) is written to x2
- **Jump Target Calculation:** PC + (-12) = 0x08
- **PC Update:** PC changes to 0x08 (jump back to LW instruction)
- **Control Signals:** RegWrite is HIGH, Jump is HIGH, PCSrc is HIGH, indicating proper JAL execution

9 Summary and Conclusion

9.1 Overall Simulation Results

The complete simulation waveform for the single-cycle RISC-V processor execution is shown below. This waveform captures the entire instruction sequence from the instruction memory, demonstrating the execution of all five instruction types (R-type, LW, SW, BEQ, and JAL) in sequence.

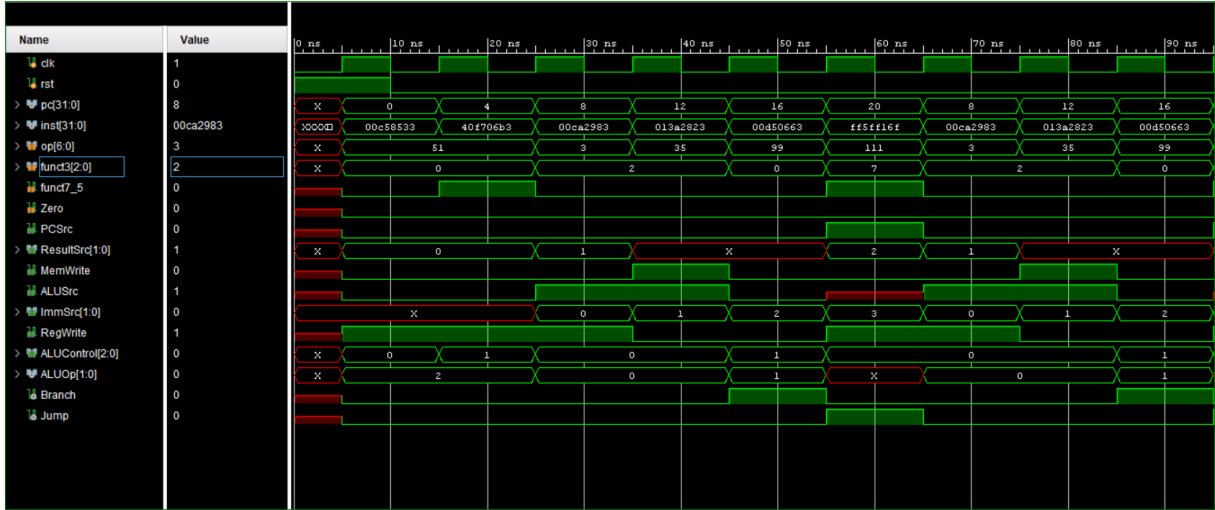


Figure 7: Complete Simulation Waveform for All Instruction Types

The overall simulation waveform demonstrates the complete execution flow:

- **PC Progression:** Program counter advances from 0 to 20 (0x00 to 0x14), then jumps back to 8 (0x08) via the JAL instruction
- **Instruction Fetch:** Each instruction is fetched from memory at the corresponding PC address
- **Control Signal Variations:** Different control signals become active or inactive based on the instruction type being executed
- **Register and Memory Operations:** Read and write operations occur at appropriate times for each instruction
- **Branch and Jump Behavior:** BEQ evaluates condition (not taken), JAL performs unconditional jump back to 8 (0x08)

The waveform clearly shows how the control unit generates the appropriate signals for each instruction type, enabling correct datapath operation. The sequence of instructions executed is:

1. ADD x10, x11, x12 at PC=0 (0x00)
2. SUB x13, x14, x15 at PC=4 (0x04)
3. LW x19, 12(x20) at PC=8 (0x08)
4. SW x19, 16(x20) at PC=12 (0x0C)
5. BEQ x10, x13, +12 at PC=16 (0x10) (not taken)
6. JAL x2, -12 at PC=20 (0x14) (jumps back to 8/0x08)

9.2 Signal State Summary by Instruction Type

The following table summarizes the control signal states for each instruction type during execution:

Signal	R-Type	LW	SW	BEQ	JAL
RegWrite	1	1	0	0	1
ALUSrc	0	1	1	0	X
MemWrite	0	0	1	0	0
PCSrc	0	0	0	0	1
Branch	0	0	0	1	0
Jump	0	0	0	0	1
ResultSrc	00	01	XX	XX	10
ImmSrc	XX	00	01	10	11
ALUControl	000/001	000	000	100	XXX
ALUOp	10	00	00	01	XX

Table 21: Control Signal States by Instruction Type

9.3 Key Observations

From the simulation results and signal analysis, the following key observations can be made:

- **RegWrite** is 1 for instructions that write to registers (R-type, LW, JAL) and 0 for store and branch instructions
- **ALUSrc** is 1 when the ALU needs an immediate value (LW, SW) and 0 when it needs register values (R-type, BEQ)
- **MemWrite** is 1 only for store instructions (SW)
- **PCSrc** is 1 for jump instructions (JAL) and branch instructions when the condition is met (BEQ)
- **ResultSrc** selects the appropriate source for write-back data
- **ImmSrc** selects the immediate format based on instruction type
- **ALUControl** determines the ALU operation based on the instruction type and function
- **Branch** is only 1 for branch instructions (BEQ)
- **Jump** is only 1 for jump instructions (JAL)

9.4 Conclusion

The single-cycle RISC-V processor successfully implements five fundamental instruction types with correct control signal generation and datapath operation. Each instruction type has unique control signal patterns that enable proper execution and data flow through the processor. The implementation demonstrates:

1. **Correct Instruction Execution:** All five instruction types execute correctly with proper datapath flow
2. **Proper Control Signal Generation:** The control unit generates appropriate signals for each instruction type
3. **Accurate Immediate Generation:** The immediate generator correctly handles different immediate formats
4. **Proper Register and Memory Operation:** Register file and memory operations work as expected
5. **Sequential and Branch Execution:** PC updates correctly for sequential and branch instructions

Benefits of RISC-V in Applications

The widespread adoption of RISC-V processors across these domains is driven by several key advantages:

- **Open-Source Architecture:** No licensing fees, enabling cost-effective solutions
- **Modular Design:** Customizable extensions for specific application requirements
- **Power Efficiency:** Optimized for low-power operation in battery-powered devices
- **Security:** Built-in security features for trusted execution environments
- **Scalability:** From tiny microcontrollers to high-performance computing systems
- **Ecosystem Support:** Growing toolchains, operating systems, and software libraries

This project serves as a foundational building block for understanding RISC-V processor design and implementation, providing the knowledge needed to develop custom processors for these diverse real-world applications.

References

1. Harris, D. M., & Harris, S. L. (2012). *Digital Design and Computer Architecture* (2nd ed.). Morgan Kaufmann.
2. Waterman, A., & Asanović, K. (2017). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. RISC-V Foundation.
3. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann.